

Inventory Management System Internal Developer Platform



AI-Driven Cloud & DevSecOps Capstone · group1-advanced · Role: Platform Engineer

Six problems slowing every Acme Retail team

The IMS was ready to ship, but how teams build and ship code was inconsistent, manual and hard to scale

Duplicate pipelines

Every team hand-writes its own CI/CD YAML

Duplicate Terraform

VPC, ECS and RDS code copied repo to repo

Different repo layouts

No shared structure, so every repo must be learned anew

Inconsistent AI specs

AI agents get different, ad-hoc instructions

Long onboarding

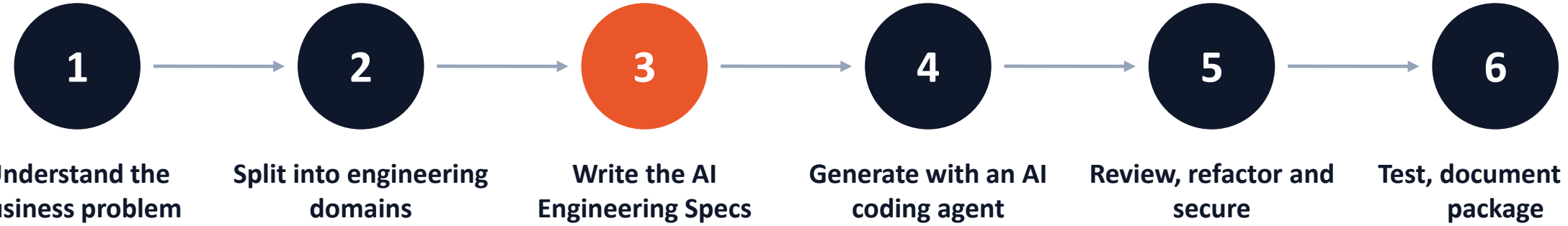
New services start from a blank page

High maintenance

Fixes must be repeated in every copy

Spec-first, AI-assisted engineering workflow

AI Engineering Specifications are the primary artifact: AI agents generate from them, engineers validate



Validated, not just generated. Every AI-generated artifact went through tests, linting, YAML parsing and security scans before it was accepted. Real Trivy findings from CI were caught and fixed this way (recorded in ADR-0004).

One platform repo, reused by every team

Six reusable building blocks replace per-team copies

Reusable workflows

CI, security scan and Terraform workflows called via workflow_call

Terraform modules

networking, ecr, ecs-fargate-service, rds-postgres, s3-bucket, iam-app-role

Repo template

Flask skeleton, caller workflows, infra and spec stubs ready to fill

Governance

CODEOWNERS, PR template, SECURITY.md and gated security scans

Self-service CLI

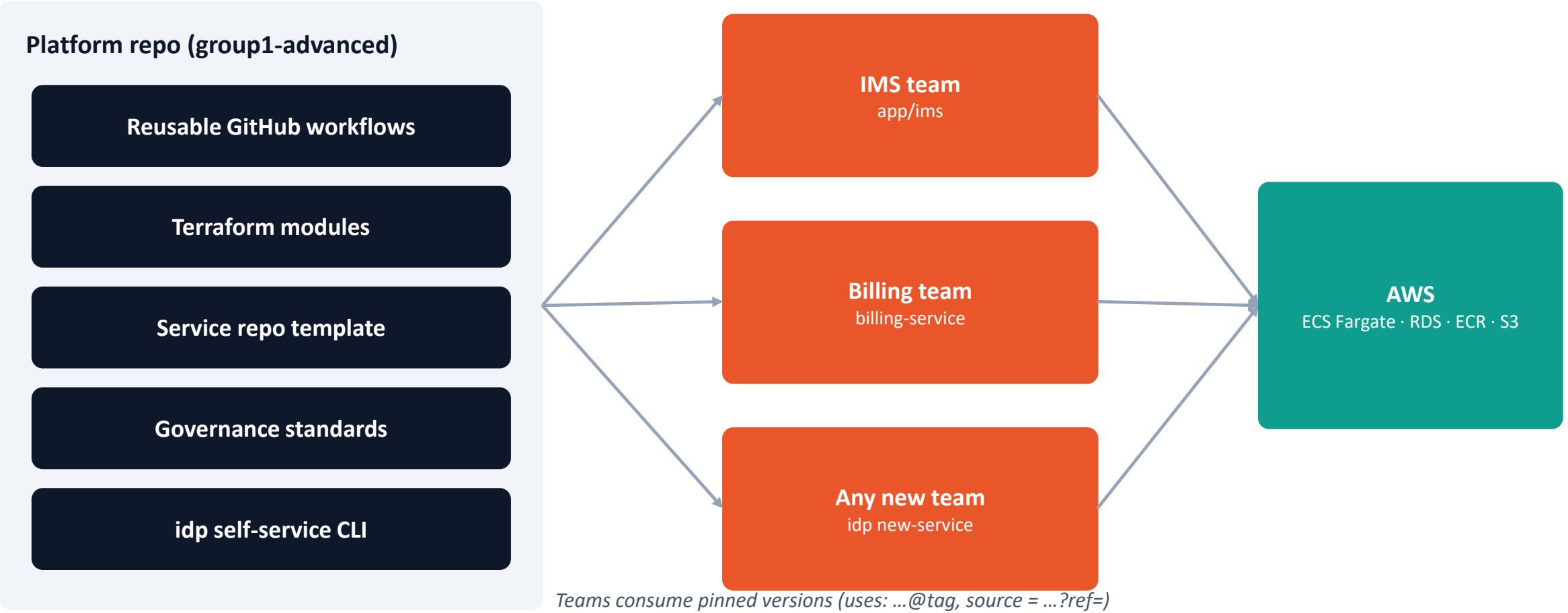
idp: scaffold, bootstrap, test, run, scan and validate in one tool

AI Engineering Specs

6 standard specs: the single source of truth for AI agents

Platform architecture: build once, reuse everywhere

App teams keep only their code and a few lines of config; everything else is referenced from the platform



Teams consume pinned versions (uses: …@tag, source = …?ref=)

Reference app: Inventory Management System

A real product that proves the platform end to end: Python 3.12 · Flask · SQLAlchemy · PostgreSQL · gunicorn

Products

CRUD + PATCH, search, filter, sort, pagination

Cart & checkout

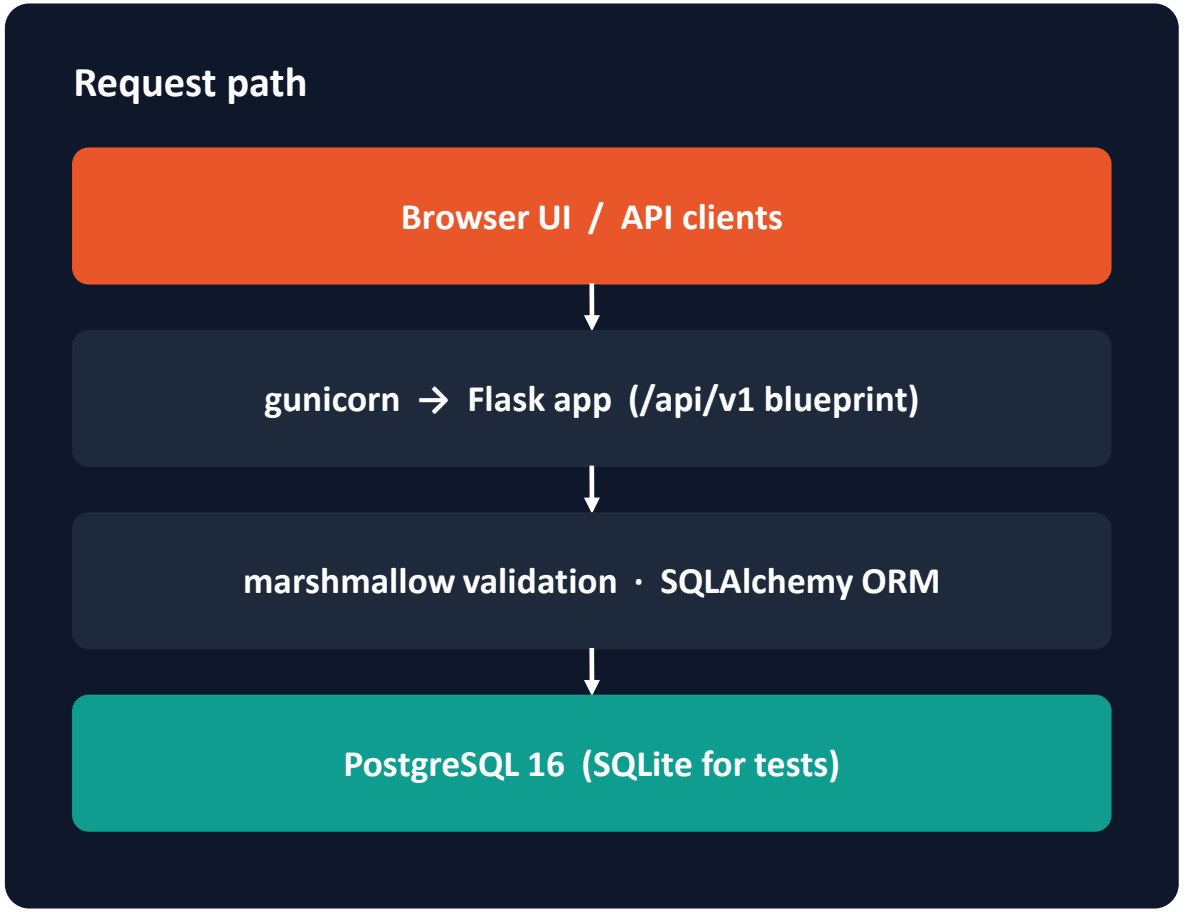
Stock-checked cart; checkout decrements stock

Orders & stats

Order history, inventory value, low-stock alerts

API-key protection

X-API-Key required for product writes



One reusable CI/CD pipeline with security gates

A caller workflow is ~20 lines; every stage lives in the platform's reusable workflows



reusable-ci.yml

reusable-security-scan.yml

reusable-terraform.yml

Shared by every team

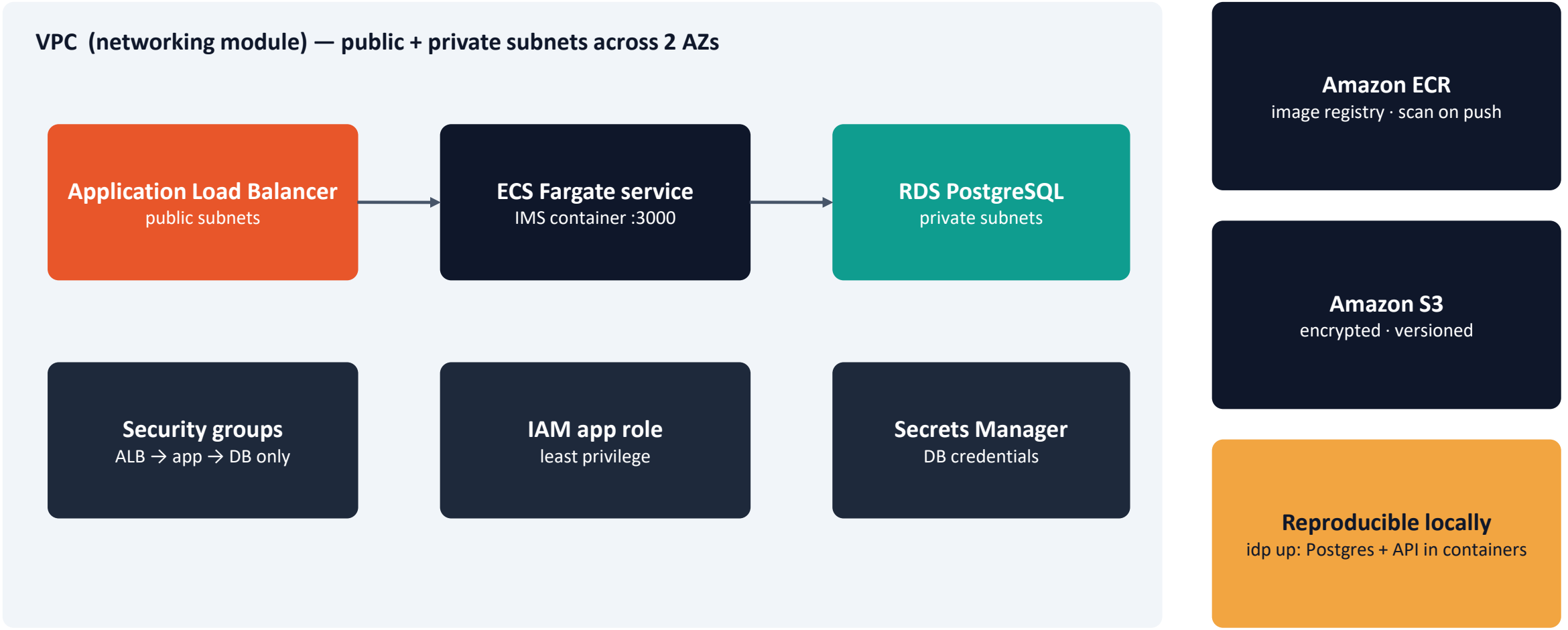
- Inputs: app_dir, python_version, image_name
- Fix a stage once and every caller gets it
- SARIF reports kept as build artifacts

Same gates locally

- idp ci → lint, test, build and scan
- idp scan → Gitleaks, Trivy, Checkov in containers
- idp tf-validate → fmt + validate on every module

AWS target architecture from shared modules

infrastructure/environments/dev composes six reusable Terraform modules



DevSecOps: security shifted left at every layer

Layer	Threat	Control	Where enforced
Code	Leaked credentials	Gitleaks secret scan	CI + idp scan
Dependencies	Known CVEs	Trivy filesystem scan, pinned versions	CI + idp scan
Container	Vulnerable / over-privileged image	Trivy image scan, Alpine, non-root, pip removed	CI + Dockerfile
Infrastructure	Cloud misconfiguration	Checkov IaC scan, encrypted S3/RDS	CI + idp scan
Change control	Unreviewed changes	CODEOWNERS, PR template	GitHub
Runtime	Unauthorised writes, exposed secrets	API key, Secrets Manager, private DB subnets	App + AWS

Two real Trivy findings surfaced in CI and were fixed — see ADR-0004

Self-service CLI: the whole platform in one command

scripts/idp.py — standard library only, runs on Windows, macOS and Linux

```
$ idp doctor
  [ok] python  [ok] git  [ok] podman
$ idp new-service billing-service --git
  Service 'billing-service' scaffolded
$ idp ci --skip-containers
  ruff: All checks passed!
  6 passed · Local CI pipeline passed.
$ idp list-services
  service   app/ims
```

doctor · list-services

Check tooling, discover services

new-service

Scaffold a repo from the template

bootstrap · lint · test

venv, deps, .env; CI-identical checks

build · up · status · down

Container image and local stack

scan · tf-validate · ci

Security gates, Terraform, full pipeline

Onboarding: from blank page to working service

Before

- Copy another team's repo by hand
- Rewrite CI/CD YAML from scratch
- Copy and adapt Terraform
- Write AI instructions ad hoc
- Fix security issues late, per repo

With the platform

- idp new-service <name>
- ~20-line caller workflows reuse platform CI
- Reference versioned Terraform modules
- Fill in 6 standard spec templates
- Security gates run from the first PR

Six AI Engineering Specifications

Shared structure: purpose, requirements (MUST/SHOULD), contracts, security, acceptance criteria, agent guardrails



platform-spec.md

Overall IDP scope, components and IMS reference



terraform-modules-spec.md

Module contracts, inputs/outputs, versioning



workflow-spec.md

Reusable CI, security and Terraform workflows



repo-template-spec.md

Template layout, placeholders, onboarding



governance-spec.md

Ownership, reviews, security policy, gates



developer-experience-spec.md

idp CLI commands and local parity

Key engineering decisions (ADRs)

ADR-0001

AWS + ECS Fargate

Managed containers with no cluster to run; RDS for Postgres

ADR-0002

Reusable workflows

workflow_call over copy-paste or composite actions

ADR-0003

Repo template + CLI

Template repo plus scripted scaffolding instead of a heavy portal

ADR-0004

Security tooling

Gitleaks, Trivy and Checkov as hard CI gates

Each ADR records Context, Problem, Decision, Alternatives, Trade-offs, Consequences and Rationale

Validation results

Everything below was run for real; items not yet verified are listed as such

6/6

IMS API tests pass

1/1

Template test passes

0

ruff lint findings

100%

workflow YAML parses

Also verified

- idp new-service scaffolds with no placeholders left
- Web UI and /health served from the container
- Trivy findings from CI fixed (ADR-0004)

Pending

- terraform validate (Terraform not installed locally)
- Full CI run on GitHub after pushing the repo
- Real AWS deployment (no cloud account provided)



Thank you



Next: push to GitHub as group1-advanced, run CI on GitHub Actions, onboard a second team